Advanced Java Programming

Module 5

Java applets

Introduction

- Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

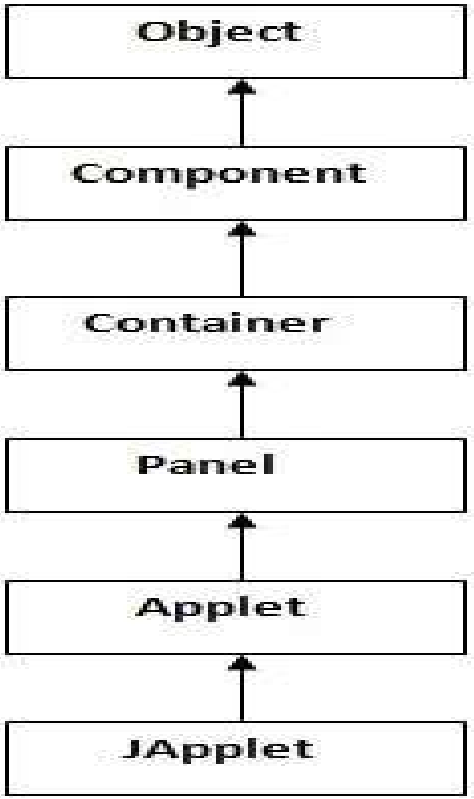
- There are many advantages of applet. They are as follows:
- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac OS etc.

Drawback of Applet

- Plugin is required at client browser to execute applet.

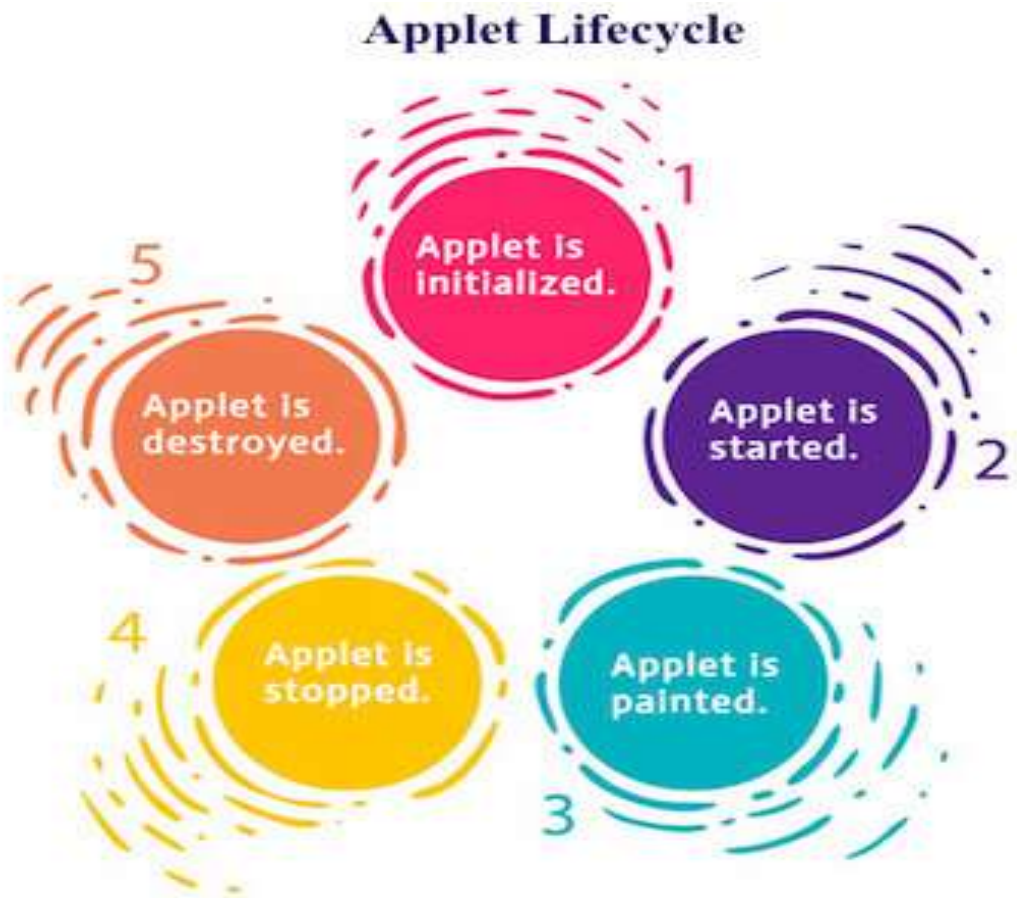
Hierarchy of Applet

- Applet class extends Panel. Panel class extends Container which is the subclass of Component.



Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.



Lifecycle methods for Applet:

- The java.applet.Applet class 4 life cycle methods and java.awt.Component class provides 1 life cycle methods for an applet.

java.applet.Applet class

- For creating any applet java.applet.Applet class must be inherited. It provides 4 life cycle methods of applet.
  1. public void init(): is used to initialized the Applet. It is invoked only once.
  2. public void start(): is invoked after the init() method or browser is maximized. It is used to start the Applet.
  3. public void stop(): is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.
  4. public void destroy(): is used to destroy the Applet. It is invoked only once.

Lifecycle methods for Applet:

java.awt.Component class

- The Component class provides 1 life cycle method of applet.
  1. public void paint(Graphics g): is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Who is responsible to manage the life cycle of an applet?

- Java Plug-in software.

How to run an Applet?

There are two ways to run an applet

1. By html file.
2. By appletViewer tool (for testing purpose).

Simple example of Applet by html file:

To execute the applet by html file, create an applet and compile it. After that create an html file and place the applet code in html file. Now click the html file.

//First.java

```
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet{
  public void paint(Graphics g){
g.drawString("welcome",150,150);
}
}
```

myapplet.html

```
<html>
<body>
<applet code="First.class" width="300"
height="300">
</applet>
</body>
</html>
```

Simple example of Applet by appletviewer tool:

To execute the applet by appletviewer tool, create an applet that contains applet tag in comment and compile it. After that run it by: appletviewer First.java. Now Html file is not required but it is for testing purpose only.

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet{
public void paint(Graphics g){
g.drawString("welcome to applet",150,150);
}
}
/*
<applet code="First.class" width="300" height="300"
>
</applet>
*/
```

To execute the applet by appletviewer tool, write in command prompt:

```
c:\>javac First.java
c:\>appletviewer First.java
```

Displaying Graphics in Applet

- Commonly used methods of Graphics class:
- public abstract void drawString(String str, int x, int y): is used to draw the specified string.
- public void drawRect(int x, int y, int width, int height): draws a rectangle with the specified width and height.
- public abstract void fillRect(int x, int y, int width, int height): is used to fill rectangle with the default color and specified width and height.
- public abstract void drawOval(int x, int y, int width, int height): is used to draw oval with the specified width and height.
- public abstract void fillOval(int x, int y, int width, int height): is used to fill oval with the default color and specified width and height.
- public abstract void drawLine(int x1, int y1, int x2, int y2): is used to draw line between the points(x1, y1) and (x2, y2).

Displaying Graphics in Applet

- public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer): is used draw the specified image.
- public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle): is used draw a circular or elliptical arc.
- public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle): is used to fill a circular or elliptical arc.
- public abstract void setColor(Color c): is used to set the graphics current color to the specified color.
- public abstract void setFont(Font font): is used to set the graphics current font to the specified font.

Example of Graphics in applet:

```
// GraphicsDemo.java
import java.applet.Applet;
import java.awt.*;
public class GraphicsDemo extends Applet{
public void paint(Graphics g){
g.setColor(Color.red);
g.drawString("Welcome",50, 50);
g.drawLine(20,30,20,300);
g.drawRect(70,100,30,30);
g.fillRect(170,100,30,30);
g.drawOval(70,200,30,30);
g.setColor(Color.pink);
g.fillOval(170,200,30,30);
g.drawArc(90,150,30,30,30,270);
g.fillArc(270,150,30,30,0,180);
}
}
```

myapplet.html

```
<html>
<body>
<applet code="GraphicsDemo.class"
width="300" height="300">
</applet>
</body>
</html>
```

Example of displaying image in applet:

Other required methods of Applet class to display image:

1.public URL getDocumentBase(): is used to return the URL of the document in which applet is embedded.

2.public URL getCodeBase(): is used to return the base URL.

```
import java.awt.*;
import java.applet.*;
public class DisplayImage extends Applet {
  Image picture;
  public void init() {
  picture =
getImage(getDocumentBase(),"sonoo.jpg");
  }
 public void paint(Graphics g) {
    g.drawImage(picture, 30,30, this);
  }
  }
```

myapplet.html

```
<html>
<body>
<applet code="DisplayImage.class"
width="300" height="300">
</applet>
</body>
</html>
```

Animation in Applet

```
// AnimationExample.java
import java.awt.*;
import java.applet.*;
public class AnimationExample extends Applet {

  Image picture;

  public void init() {
    picture
=getImage(getDocumentBase(),"bike_1.gif");
  }
    public void paint(Graphics g) {
    for(int i=0;i<500;i++){
      g.drawImage(picture, i,30, this);
       try{Thread.sleep(100);}catch(Exception e){}
    }
  }
}
```

myapplet.html

```
<html>
<body>
<applet code="AnimationExample.class"
width="300" height="300">
</applet>
</body>
</html>
```

EventHandling in Applet

As we perform event handling in AWT or Swing, we can perform it in applet also. Let's see the simple example of event handling in applet that prints a message by click on the button.

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
public class EventApplet extends Applet implements A
ctionListener{
Button b;
TextField tf;
public void init(){
tf=new TextField();
tf.setBounds(30,40,150,20);
b=new Button("Click");
b.setBounds(80,150,60,50);
add(b);add(tf);
b.addActionListener(this);
setLayout(null);
}
 public void actionPerformed(ActionEvent e){
  tf.setText("Welcome");
 }
}
```

myapplet.html

```
<html>
<body>
<applet code="EventApplet.class"
width="300" height="300">
</applet>
</body>
</html>
```

JApplet class in Applet

As we prefer Swing to AWT. Now we can use JApplet that can have all the controls of swing. The JApplet class extends the Applet class.

```
import java.applet.*;
import javax.swing.*;
import java.awt.event.*;
public class EventJApplet extends JApplet implements Ac
tionListener{
JButton b;
JTextField tf;
public void init(){
tf=new JTextField();
tf.setBounds(30,40,150,20);
b=new JButton("Click");
b.setBounds(80,150,70,40);
add(b);add(tf);
b.addActionListener(this);
 setLayout(null);
}
public void actionPerformed(ActionEvent e){
tf.setText("Welcome");
}
}
```

myapplet.html

```
<html>
<body>
<applet code="EventJApplet.class"
width="300" height="300">
</applet>
</body>
</html>
```

Parameter in Applet

```
import java.applet.Applet;
import java.awt.Graphics;

public class UseParam extends Applet{

public void paint(Graphics g){
String str=getParameter("msg");
g.drawString(str,50, 50);
}

}
```

We can get any information from the HTML file as a parameter. For this purpose, Applet class provides a method named getParameter(). Syntax:

public String getParameter(String parameterName)

myapplet.html

```
<html>
<body>
<applet code="UseParam.class"
width="300" height="300">
<param name="msg" value="Welcome
to applet">
</applet>
</body>
</html>
```

Applet Communication

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
public class ContextApplet extends Applet implements
ActionListener{
Button b;
public void init(){
b=new Button("Click");
b.setBounds(50,50,60,50);
add(b);
b.addActionListener(this);
}
public void actionPerformed(ActionEvent e){
AppletContext ctx=getAppletContext();
Applet a=ctx.getApplet("app2");
a.setBackground(Color.yellow);
}
}
```

java.applet.AppletContext class provides the facility of communication between applets. We provide the name of applet through the HTML file. It provides getApplet() method that returns the object of Applet. Syntax:

public Applet getApplet(String name){}

myapplet.html

```
<html>
<body>
<applet code="ContextApplet.class"
width="150" height="150"
name="app1">
</applet>
 <applet code="First.class" width="150"
height="150" name="app2">
</applet>
</body>
</html>
```

NETWORKING

NETWORKING

- InetAddress
- TCP/ IP client sockets
- TCP/ IP server sockets
- URL
- URL Connection
- Datagrams
- Client/ Server application using RMI.

NETWORKING

Networking Basics

• In Java Networking is a concept of connecting two or more computing devices together so that we can share resources.

• Java socket programming provides facility to share data between different computing devices.

Advantage of Java Networking

1. Sharing resources
2. Centralize software management

NETWORKING

Java Networking Terminology

- The widely used java networking terminologies are given below:
    1. IP Address
    2. Protocol
    3. Port Number
    4. MAC Address
    5. Connection-oriented and connection-less protocol
    6. Socket

NETWORKING

1) IP Address

• IP address is a unique number assigned to a node of a network e.g. 192.168.0.1.

• It is composed of octets that range from 0 to 255.

• It is a logical address that can be changed.

2) Protocol

• A protocol is a set of rules basically that is followed for communication.

• For example: TCP, FTP, Telnet, SMTP, POP etc.

3) Port Number

• The port number is used to uniquely identify different applications.

• It acts as a communication endpoint between applications.

• The port number is associated with the IP address for communication between two applications.

NETWORKING

4) MAC (Media Access Control) Address

- *MAC Address is a unique identifier of NIC (Network Interface Controller).*
- *A network node can have multiple NIC but each with unique MAC.*

5) Connection-oriented and Connection-less protocol

- In connection-oriented protocol, acknowledgement is sent by the receiver. So it is reliable but slow. The example of connection-oriented protocol is TCP.
- But, in connection-less protocol, acknowledgement is not sent by the receiver. So it is notreliable but fast. The example of connection-less protocol is UDP.

6) Socket

- A socket is an endpoint between two way communication.

NETWORKING

java.net package

- The java.net package provides many classes to deal with networking applications in Java. A list of these classes is given below:

- Authenticator
- ContentHandler
- DatagramPacket
- InterfaceAddress
- InetSocketAddress
- Inet6Address
- HttpCookie
- PasswordAuthentication
- ResponseCache
- Socket
- SocketPermission
- URL
- URLDecoder
- CacheRequest
- CookieHandler
- DatagramSocket
- JarURLConnection
- InetAddress
- IDN
- NetPermission
- Proxy
- SecureCacheResponse
- SocketAddress
- URI
- URLClassLoader
- URLEncoder
- CacheResponse
- CookieManager
- DatagramSocketImpl
- MulticastSocket
- Inet4Address
- HttpURLConnection
- NetworkInterface
- ProxySelector
- ServerSocket
- SocketImpl
- StandardSocketOptions
- URLConnection
- URLStreamHandler

NETWORKING

InetAddress class

- **InetAddress** class represents an IP address.
- The java.net.InetAddress class provides methods to get the IP of any host name *for example* www.javatpoint.com, www.google.com, www.facebook.com, etc.
- An IP address is represented by 32-bit or 128-bit unsigned number.
- An instance of InetAddress represents the IP address with its corresponding host name.
- There are two types of address types: ▪ Unicast ▪ Multicast.
- The Unicast is an identifier for a single interface whereas Multicast is an identifier for a set of interfaces.
- Moreover, InetAddress has a cache mechanism to store successful and unsuccessful host name resolutions.

NETWORKING

Methods of InetAddress class

Method	Description
public static InetAddress getByName(String host) throws UnknownHostException	it returns the instance of InetAddress containing LocalHost IP and name.
public static InetAddress getLocalHost() throws UnknownHostException	it returns the instance of InetAdddress containing local host name and address.
public String getHostName()	it returns the host name of the IP address.
public String getHostAddress()	it returns the IP address in string format.

NETWORKING

Example of InetAddress class

```
import java.io.*;
import java.net.*;
public class InetDemo{
public static void main(String[] args){
try{
InetAddress ip=InetAddress.getByName("www.javatpoint.com");
System.out.println("Host Name: "+ip.getHostName());
System.out.println("IP Address: "+ip.getHostAddress());
}catch(Exception e){System.out.println(e);}
}
}
```

Output:
Host Name: www.javatpoint.com
IP Address: 206.51.231.148

NETWORKING

Socket Programming

- Java Socket programming is used for communication between the applications running on different JRE.
- Java Socket programming can be connection-oriented or connection-less.
- Socket and ServerSocket classes are used for connection-oriented socket programming
- DatagramSocket and DatagramPacket classes are used for connection-less socket programming.
- The client in socket programming must know two information:

1. IP Address of Server
2. Port number

NETWORKING

- Here, we are going to make one-way client and server communication.
- In this application, client sends a message to the server, server reads the message and prints it. Here, two classes are being used: Socket and ServerSocket.
- The Socket class is used to communicate client and server. Through this class, we can read and write message.
- The ServerSocket class is used at server-side.
- The accept() method of ServerSocket class blocks the console until the client is connected.
- After the successful connection of client, it returns the instance of Socket at server-side.

NETWORKING

TCP/IP Client Socket

Socket class

- A socket is simply an endpoint for communications between the machines.
- The Socket class can be used to create a socket.

Methods:

Method	Description
1) public InputStream getInputStream()	returns the InputStream attached with this socket.
2) public OutputStream getOutputStream()	returns the OutputStream attached with this socket.
3) public synchronized void close()	closes this socket

NETWORKING

TCP/IP Server Socket

ServerSocket class

• The ServerSocket class can be used to create a server socket. This object is used to establish communication with the clients.

Methods:

Method	Description
1) public Socket accept()	returns the socket and establish a connection between server and client.
2) public synchronized void close()	closes the server socket.

NETWORKING

Creating Server:

• To create the server application, we need to create the instance of ServerSocket class.

• Here, we are using 6666 port number for the communication between the client and server.

• You may also choose any other port number.

• The accept() method waits for the client. If clients connects with the given port number, it returns an instance of Socket.

```
ServerSocket ss=new ServerSocket(6666);
Socket s=ss.accept();//establishes connection and waits for the client
```

NETWORKING

Creating Client:

- To create the client application, we need to create the instance of Socket class.
- Here, we need to pass the IP address or hostname of the Server and a port number.
- Here, we are using "localhost" because our server is running on same system.

```
Socket s=new Socket("localhost",6666);
```

NETWORKING

URL

- *The Java URL class represents an URL.*
- *URL is an acronym for Uniform Resource Locator. It points to a resource on the World Wide Web.*
- *For example: https://www.javatpoint.com/java-tutorial*
- *A URL contains four information:*

1. Protocol: In this case, http is the protocol.

2. Server name or IP Address: In this case, www.javatpoint.com is the server name.

3. Port Number: It is an optional attribute. If we write

http//www.javatpoint.com:80/sonoojaiswal/ , 80 is the port number. If port number is not mentioned in the URL, it returns -1.

4. File Name or directory name: In this case, index.jsp is the file name.

NETWORKING

Constructors of URL class

- ***URL(String spec)***

✓*Creates an instance of a URL from the String representation.*

- *URL(String protocol, String host, int port, String file)*

✓*Creates an instance of a URL from the given protocol, host, port number, and file.*

- ***URL(String protocol, String host, int port, String file, URLStreamHandler handler)***

✓*Creates an instance of a URL from the given protocol, host, port number, file, and handler.*

NETWORKING

- ***URL(String protocol, String host, String file)***

✓ Creates an instance of a URL from the given protocol name, host name, and file name.

- **URL(URL context, String spec)**

✓ Creates an instance of a URL by parsing the given spec within a specified context.

- **URL(URL context, String spec, URLStreamHandler handler)**

✓ Creates an instance of a URL by parsing the given spec with the specified handler within a given context.

NETWORKING

Methods of URL class

• *The java.net.URL class provides many methods. The important methods of URL class are given below.*

Methods – Description

- public String getProtocol() - It returns the protocol of the URL.
- public String getHost() - It returns the host name of the URL.
- public String getPort() - It returns the Port Number of the URL.
- public String getFile() - It returns the file name of the URL.
- public String getAuthority() - It returns the authority of the URL.
- public String toString() - It returns the string representation of the URL.
- public String getQuery() - It returns the query string of the URL.

NETWORKING

Methods – Description

- public String getDefaultPort() - It returns the default port of the URL.
- public URLConnection openConnection() - It returns the instance of URLConnection i.e. associated with this URL.
- public boolean equals(Object obj) - It compares the URL with the given object.
- public Object getContent() - It returns the content of the URL.
- public String getRef() - It returns the anchor or reference of the URL.
- public URI toURI() - It returns a URI of the URL.

NETWORKING

Example of URL class

```
//URLDemo.java
import java.net.*;
public class URLDemo{
public static void main(String[] args){
try{ URL url=new URL("http://www.javatpoint.com/java-tutorial");
System.out.println("Protocol: "+url.getProtocol());
System.out.println("Host Name: "+url.getHost());
System.out.println("Port Number: "+url.getPort());
System.out.println("File Name: "+url.getFile());
}catch(Exception e){System.out.println(e);}
}
}
```

Output:

```
Protocol: http
Host Name: www.javatpoint.com
Port Number: -1
File Name: /java-tutorial
```

NETWORKING

URLConnection class

- *The Java URLConnection class represents a communication link between the URL and the application.*
- *This class can be used to read and write data to the specified resource referred by the URL.*

Object of URLConnection class

- The openConnection() method of URL class returns the object of URLConnection class.

Syntax:public URLConnection openConnection()throws IOException{}

- The URLConnection class provides many methods, we can display all the data of a webpage by using the getInputStream() method.
- The getInputStream() method returns all the data of the specified URL in the stream that can be read and displayed.

NETWORKING

Example of URLConnection class

```
import java.io.*;
import java.net.*;
public class URLConnectionExample {
public static void main(String[] args){
try{
URL url=new URL("http://www.javatpoint.com/java-tutorial");
URLConnection urlcon=url.openConnection();
InputStream stream=urlcon.getInputStream();
int i;
while((i=stream.read())!=-1){
System.out.print((char)i); }
}catch(Exception e){System.out.println(e);}
} }
```

NETWORKING

HttpURLConnection class

- *The Java HttpURLConnection class is http specific URLConnection. It works for HTTP protocol only.*
- *By the help of HttpURLConnection class, you can information of any HTTP URL such as header information, status code, response code etc.*
- *The java.net.HttpURLConnection is subclass of URLConnection class.*

Object of HttpURLConnection class

- The openConnection() method of URL class returns the object of URLConnection class.

Syntax:public URLConnection openConnection()throws IOException{}

- You can typecast it to HttpURLConnection type as given below.

```
URL url=new URL("http://www.javatpoint.com/java-tutorial");
HttpURLConnection huc=(HttpURLConnection)url.openConnection();
```

NETWORKING

HttpURLConnecton Example

```
import java.io.*;
import java.net.*;
public class HttpURLConnectionDemo{
public static void main(String[] args){
try{
URL url=new URL("http://www.javatpoint.com/java-tutorial");
HttpURLConnection huc=(HttpURLConnection)url.openConnection();
for(int i=1;i<=8;i++){
System.out.println(huc.getHeaderFieldKey(i)+" = "+huc.getHeaderField(i));
}
huc.disconnect();
}catch(Exception e){System.out.println(e);}
} }
```

Output:

```
Date = Wed, 10 Dec 2014 19:31:14 GMT
Set-Cookie =
JSESSIONID=D70B87DBB832820CACA5998C90939D48;
Path=/
Content-Type = text/html
Cache-Control = max-age=2592000
Expires = Fri, 09 Jan 2015 19:31:14 GMT
Vary = Accept-Encoding,User-Agent
Connection = close
Transfer-Encoding = chunked
```

NETWORKING

Datagrams

- *Datagrams are bundles of information passed between machines.*
- *Java DatagramSocket and DatagramPacket classes are used for connection-less socket programming.*

DatagramSocket class

- *Java DatagramSocket class represents a connection-less socket for sending and receiving datagram packets.*
- *A datagram is basically an information but there is no guarantee of its content, arrival or arrival time.*

NETWORKING

Constructors of DatagramSocket class

- *DatagramSocket() throws SocketException: it creates a datagram socket and binds it with the available Port Number on the localhost machine.*
- *DatagramSocket(int port) throws SocketException: it creates a datagram socket and binds it with the given Port Number.*
- *DatagramSocket(int port, InetAddress address) throws SocketException: it creates a datagram socket and binds it with the specified port number and host address.*

DatagramPacket class

- *Java DatagramPacket is a message that can be sent or received. If you send multiple packet, it may arrive in any order. Additionally, packet delivery is not guaranteed*

NETWORKING

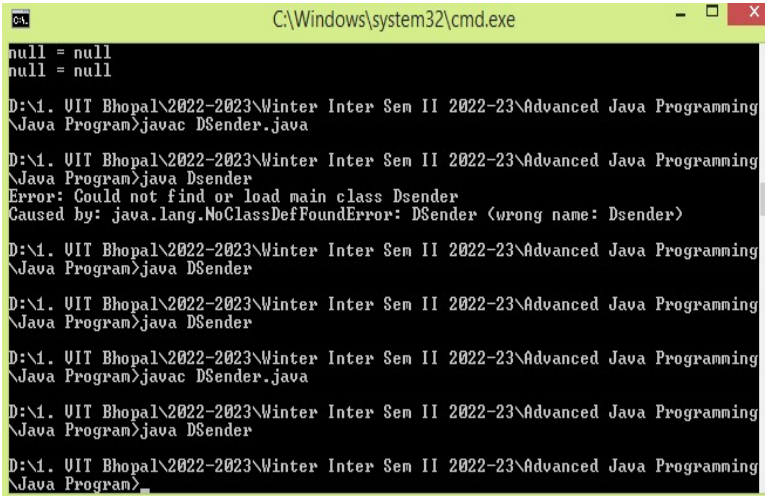
Constructors of DatagramPacket class

- *DatagramPacket(byte[] barr, int length): it creates a datagram packet. This constructor is used to receive the packets.*
- *DatagramPacket(byte[] barr, int length, InetAddress address, int port): it creates a datagram packet. This constructor is used to send the packets.*

NETWORKING

Example of Sending DatagramPacket by DatagramSocket

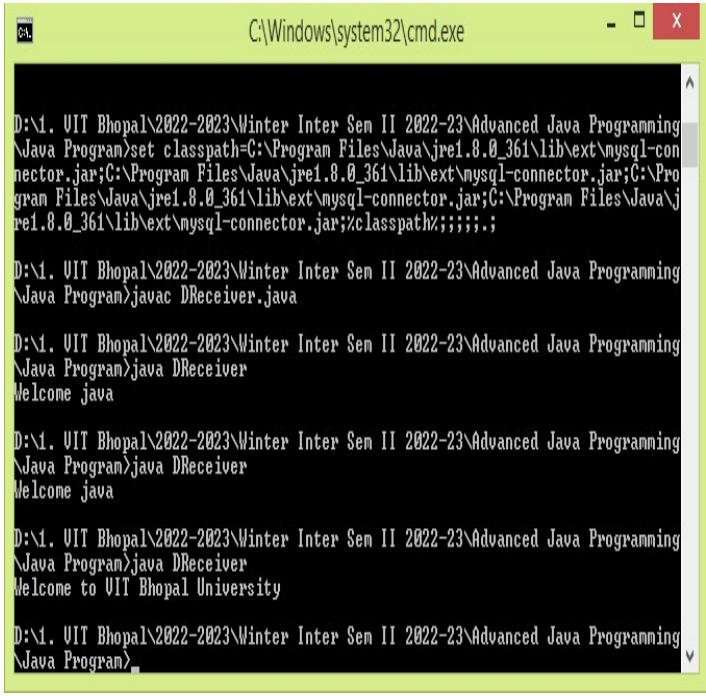
```
//DSender.java
import java.net.*;
public class DSender{
public static void main(String[] args) throws Exception {
DatagramSocket ds = new DatagramSocket();
String str = "Welcome to VIT Bhopal University";
InetAddress ip = InetAddress.getByName("127.0.0.1");
DatagramPacket dp = new DatagramPacket(str.getBytes(), str.length(), ip, 3000);
ds.send(dp);
ds.close();
}
}
```



NETWORKING

Example of Receiving DatagramPacket by DatagramSocket

```
//DReceiver.java
import java.net.*;
public class DReceiver{
public static void main(String[] args) throws Exception {
DatagramSocket ds = new DatagramSocket(3000);
byte[] buf = new byte[1024];
DatagramPacket dp = new DatagramPacket(buf, 1024);
ds.receive(dp);
String str = new String(dp.getData(), 0, dp.getLength());
System.out.println(str);
ds.close();
} }
```



RMI(Remote Method Invocation)

RMI

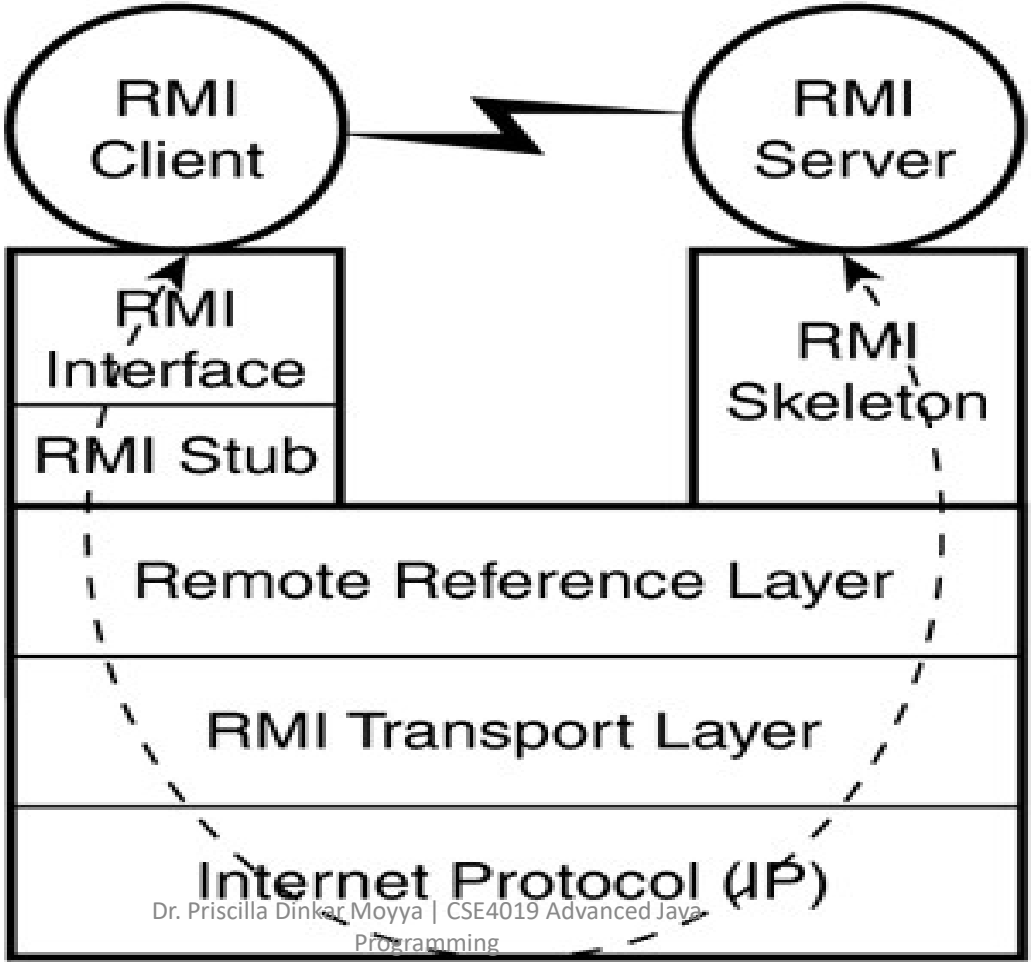
Client Server Application using RMI (Remote Method Invocation)

- *The RMI (Remote Method Invocation) is an API that provides a mechanism to create distributed application in java.*
- *The RMI allows an object to invoke methods on an object running in another JVM.*
- *The RMI provides remote communication between the applications using two objects stub and skeleton.*
- *RMI uses stub and skeleton object for communication with the remote object.*
- *A remote object is an object whose method can be invoked from another JVM.*

RMI – Functionalities’

- **Remote communication**: RMI enables remote communication between Java programs.
- **Object serialization**: RMI uses object serialization to marshal and unmarshal parameters.
- **Object transmission**: RMI can transmit the types and behavior of an object to another JVM.
- **Object class definition download**: RMI can download the definition of an object's class if it's not defined in the receiver's JVM.
- **Object behavior movement**: RMI can move behavior (class implementations) from client to server and vice versa.
- **True object-oriented polymorphism**: RMI supports true object-oriented polymorphism by not truncating types

RMI Architecture



RMI

- The Remote Reference Layer (RRL) and Transport Connection Layer are two of the layers in Java's Remote Method Invocation (RMI):
  - **Remote Reference Layer (RRL)**: Manages the references made by the client to the remote object on the server.
    - The RRL determines whether the server is a single object or a replicated object. It also abstracts the different ways of referring to objects.
  - **Transport Layer** – This layer connects the client and the server. It manages the existing connection and also sets up new connections.

RMI

Stub

- *The stub is an object, acts as a gateway for the client side.*
- *All the outgoing requests are routed through it.*
- *It resides at the client side and represents the remote object.*
- *When the caller invokes method on the stub object, it does the following tasks:*
  1. *It initiates a connection with remote Virtual Machine (JVM),*
  2. *It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),*
  3. *It waits for the result*
  4. *It reads (unmarshals) the return value or exception, and*
  5. *It finally, returns the value to the caller.*

RMI

Skeleton

- The skeleton is an object, acts as a gateway for the server side object.
- All the incoming requests are routed through it.
- When the skeleton receives the incoming request, it does the following tasks:
    1. It reads the parameter for the remote method
    2. It invokes the method on the actual remote object, and
    3. It writes and transmits (marshals) the result to the caller.
- In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.
- If any application performs these tasks, it can be distributed application.
    1. The application need to locate the remote method
    2. It need to provide the communication with the remote objects, and
    3. The application need to load the class definitions for the objects.

The RMI application has all these features, so it is called the distributed application.

RMI Example

The following 6 steps given are to write the RMI program:

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the rmic tool
4. Start the registry service by rmiregistry tool
5. Create and start the remote application
6. Create and start the client application

- In this example, we have followed all the 6 steps to create and run the rmi application.
- The client application need only two files, remote interface and client application.
- In the rmi application, both client and server interacts with the remote interface.
- The client application invokes methods on the proxy object, RMI sends the request to the remote JVM.
- The return value is sent back to the proxy object and then to the client application.

RMI Example

1) Create the remote interface

• For creating the remote interface, extend the Remote interface and declare the RemoteException with all the methods of the remote interface.

• Here, we are creating a remote interface that extends the Remote interface.

• There is only one method named add() and it declares RemoteException.

```
import java.rmi.*;
public interface Adder extends Remote{
public int add(int x,int y)throws RemoteException;
}
```

RMI Example

2) Provide the implementation of the remote interface

• Now provide the implementation of the remote interface. For providing the implementation of the Remote interface, we need to

- o Either extend the UnicastRemoteObject class, (or)
- o Use the exportObject() method of the UnicastRemoteObject class

• In case, you extend the UnicastRemoteObject class, you must define a constructor that declares RemoteException.

```
import java.rmi.*;
import java.rmi.server.*;
public class AdderRemote extends UnicastRemoteObject implements Adder{
AdderRemote()throws RemoteException{
super(); }
public int add(int x,int y){return x+y;}
}
```

RMI Example

3) Create the stub and skeleton objects using the rmic tool

- Next step is to create stub and skeleton objects using the rmi compiler.
- The rmic tool invokes the RMI compiler and creates stub and skeleton objects.

rmic AdderRemote

4) Start the registry service by the rmiregistry tool

- Now start the registry service by using the rmiregistry tool.
- If you don't specify the port number, it uses a default port number.
- In this example, we are using the port number 5000.

rmiregistry 5000

RMI Example

5) Create and run the server application

- Now rmi services need to be hosted in a server process.
- The Naming class provides methods to get and store the remote object.
- The Naming class provides 5 methods.

public static java.rmi.Remote lookup(java.lang.String) throws java.rmi.NotBoundException, java.net.MalformedURLException, java.rmi.RemoteException;	It returns the reference of the remote object.
public static void bind(java.lang.String, java.rmi.Remote) throws java.rmi.AlreadyBoundException, java.net.MalformedURLException, java.rmi.RemoteException;	It binds the remote object with the given name.

RMI Example

public static void unbind(java.lang.String) throws java.rmi.RemoteException, java.rmi.NotBoundException, java.net.MalformedURLException;	It destroys the remote object which is bound with the given name.
public static void rebind(java.lang.String, java.rmi.Remote) throws java.rmi.RemoteException, java.net.MalformedURLException;	It binds the remote object to the new name.
public static java.lang.String[] list(java.lang.String) throws java.rmi.RemoteException, java.net.MalformedURLException;	It returns an array of the names of the remote objects bound in the registry.

RMI Example

In this example, we are binding the remote object by the name sonoo.

```
import java.rmi.*;
import java.rmi.registry.*;
public class MyServer{
public static void main(String args[]){
try{
Adder stub=new AdderRemote();
Naming.rebind("rmi://localhost:5000/sonoo",stub);
}catch(Exception e){System.out.println(e);}
}
}
```

RMI Example

6) Create and run the client application

• At the client we are getting the stub object by the lookup() method of the Naming class and invoking the method on this object.

• In this example, we are running the server and client applications, in the same machine so we are using localhost.

• If you want to access the remote object from another machine, change the localhost to the host name (or IP address) where the remote object is located.

```
import java.rmi.*;
public class MyClient{
public static void main(String args[]){
try{
Adder stub=(Adder)Naming.lookup("rmi://localhost:5000/sonoo");
System.out.println(stub.add(34,4));
}catch(Exception e){}
} }
```

RMI Example

For running this rmi example,

1. Compile all the java files: javac *.java
2. Create stub and skeleton object by rmic tool: rmic AdderRemote
3. Start rmi registry in one command prompt: rmiregistry 5000
4. Start the server in another command prompt: java MyServer
5. Start the client application in another command prompt: java MyClient
6. Display the Output

Server-side Programming: Java Servlets

What is a Servlet?

- Servlet is a technology i.e. used to create web application.
- Servlet is an API that provides many interfaces and classes including documentations.
- Servlet is a class that extends the capabilities of the servers and responds to the incoming requests. It can respond to any type of requests.
- Servlet is a web component that is deployed on the server to create dynamic web page.

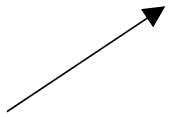
Java Servlet

- Javax.servlet package can be extended for use with any application layer protocol
- http is the most popularly used protocol
- Javax.servlet.http package is extension of the javax.servlet package for http protocol
- The Servlet spec allows you to implement separate Java methods implementing each HTTP method in your subclass of HttpServlet.
- Override the doGet() and/or doPost() method to provide normal servlet functionality.
- Override doPut() or doDelete() if you want to implement these methods.
- There's no need to override doOptions() or doTrace().
- The superclass handles the HEAD method all on its own.

Servlets - What are they?

- A servlet is a Java object which resides within a servlet engine. A servlet engine is usually contained within a web server.
- Servlets are Java objects which respond to HTTP requests. Servlets may return data of any type but they often return HTML.
- Servlets are invoked through a URL which means that a servlet can be invoked from a browser.
- Servlets can be passed parameters via the HTTP request.

http://www.somehost.com/servlet/search?word=Java&language=english



Keyword “servlet” indicates to web server that this request is for a servlet.

Servlet called search

Parameters encoded in an HTTP request

Getting Started

- All servlets must import the following packages:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
```

- All servlets must extend the HttpServlet or Servlet class

```
public class MyServlet extends HttpServlet
```

- Servlets must provide and implementation for doGet, doPost or both.

```
public void doGet(HttpServletRequest request,
                  HttpServletResponse response) throws
                  ServletException, IOException
{
        // ...
```

Generating Output

- Set the content type of the return message. NOTE: this must be set before any data is returned from the request

```
response.setContentType("text/html");
```

- HTML can be written as text to the response. Obtain the PrintWriter object from the response and write text to it

```
PrintWriter out = response.getWriter();
out.println("<HTML><HEAD><TITLE>Test</TITLE>");
out.println("<BODY>><H1>My FirstServlet</H1>");
out.println("</BODY></HTML>");
out.close();
```

doPost and doGet

- Under HTTP, there are two methods available for sending parameters from the browser to the web server. They are POST and GET
- POST and GET are virtually the same except in terms of how parameter data is passed

- GET: Parameters are encoded within the URL. If data is passed via GET, it is limited in size to 2K
- POST: Parameters are sent AFTER the HTTP header in the request. There is no limit to the size of parameters sent via POST.

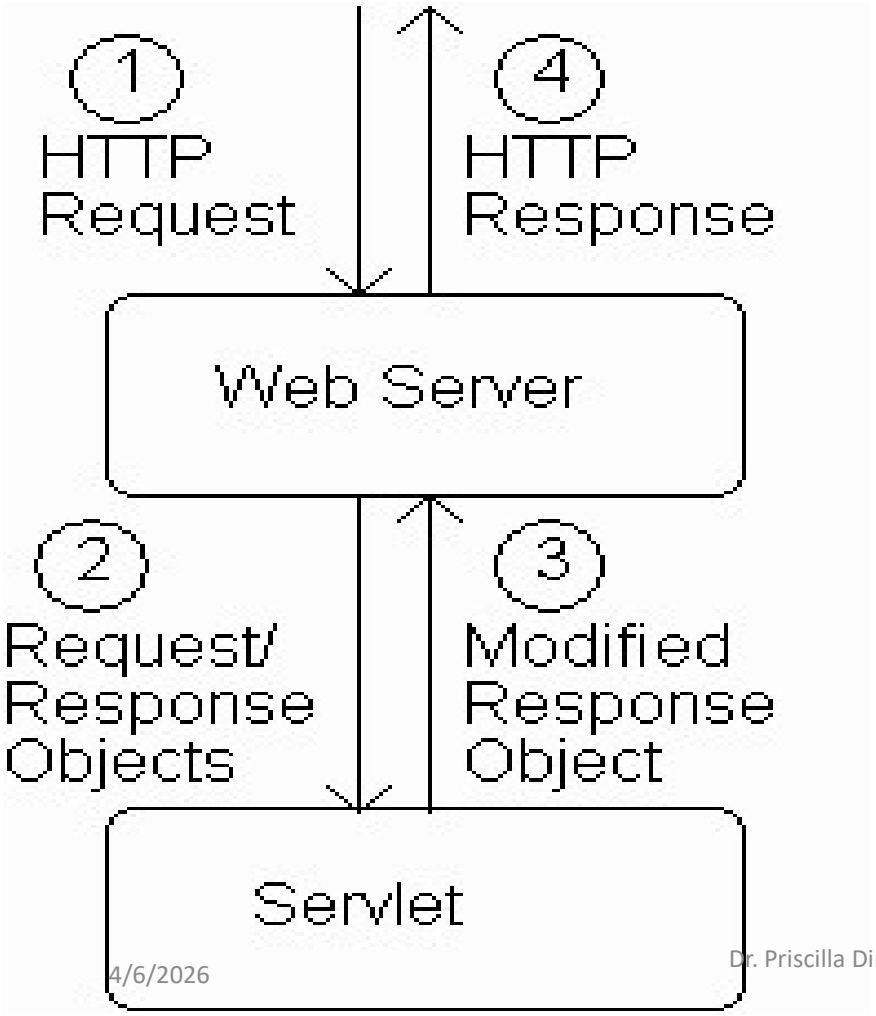
- The method of parameter passing is defined in the HTML loaded into the browser. Servlet authors can choose to implement the doGet method, doPost method or both.

Servlet Life Cycle

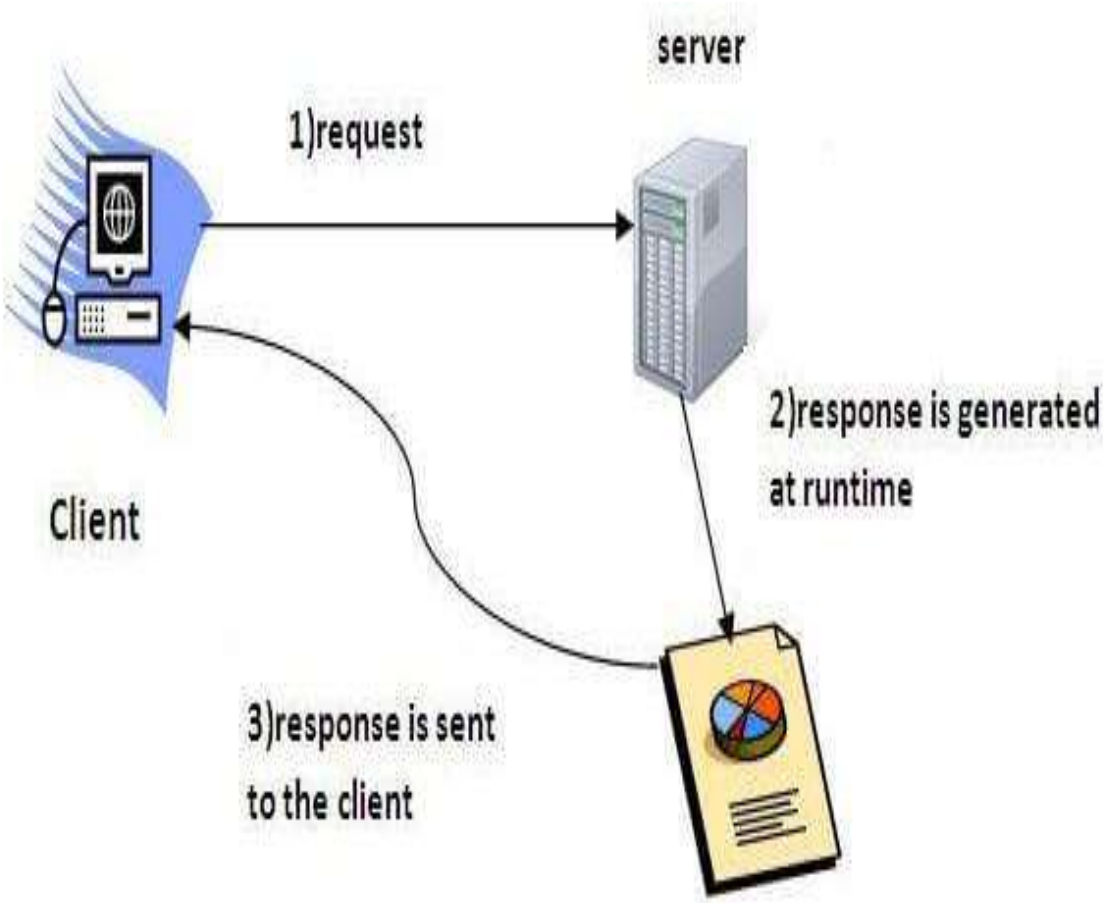
- When a servlet is FIRST requested, it is loaded into the servlet engine. The init() method of the servlet is invoked so that the servlet may initialize itself.
- Once initialization is complete, the request is then forwarded to the appropriate method (ie. doGet or doPost)
- The servlet is then held in memory. Subsequent requests are simply forwarded to the servlet object.
- When the engine wishes to remove the servlet, its destroy() method is invoked.

- NOTE: Servlets can receive multiple requests for multiple clients at any given time. Therefore, servlets must be thread safe

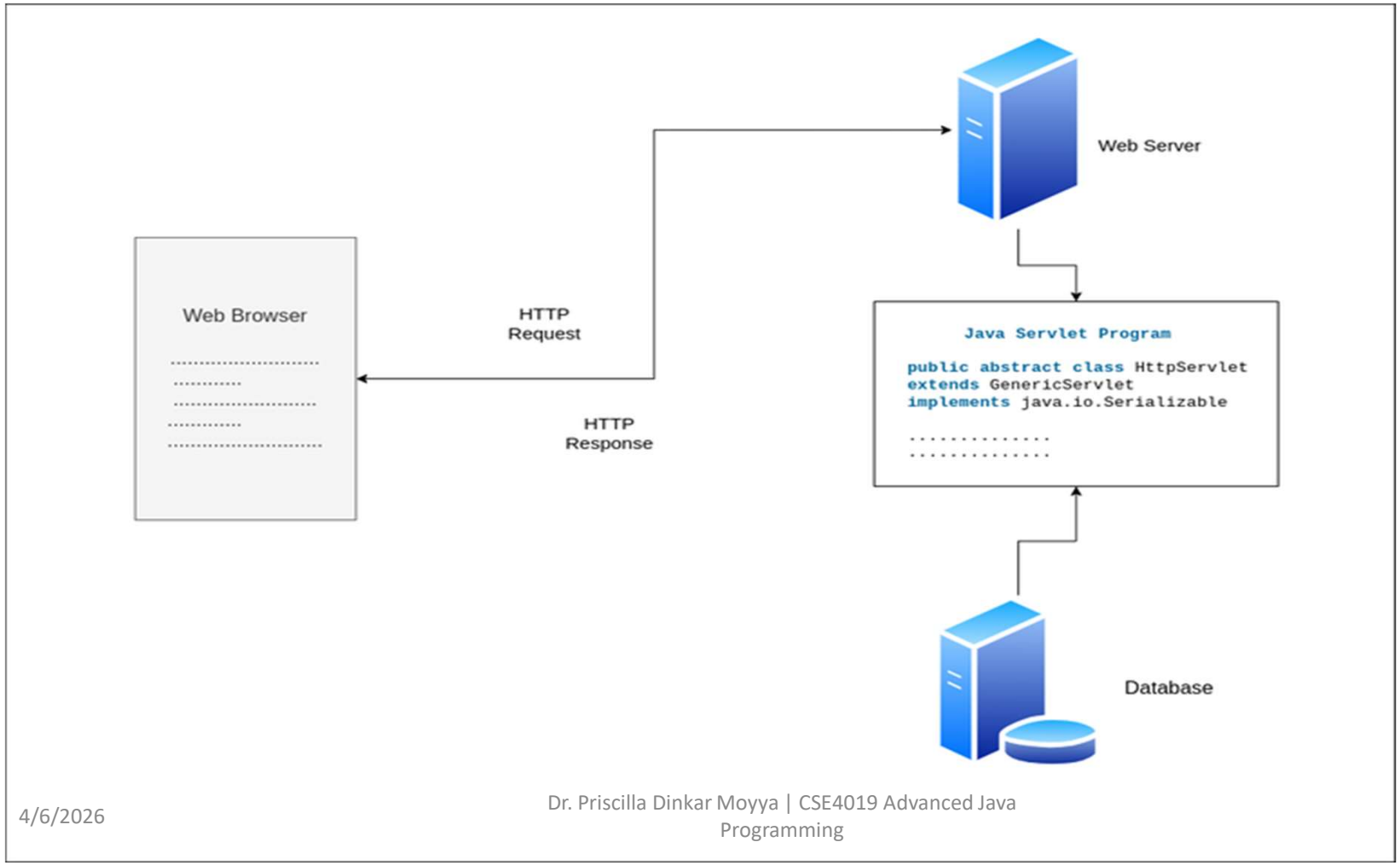
Architecture Diagram 1



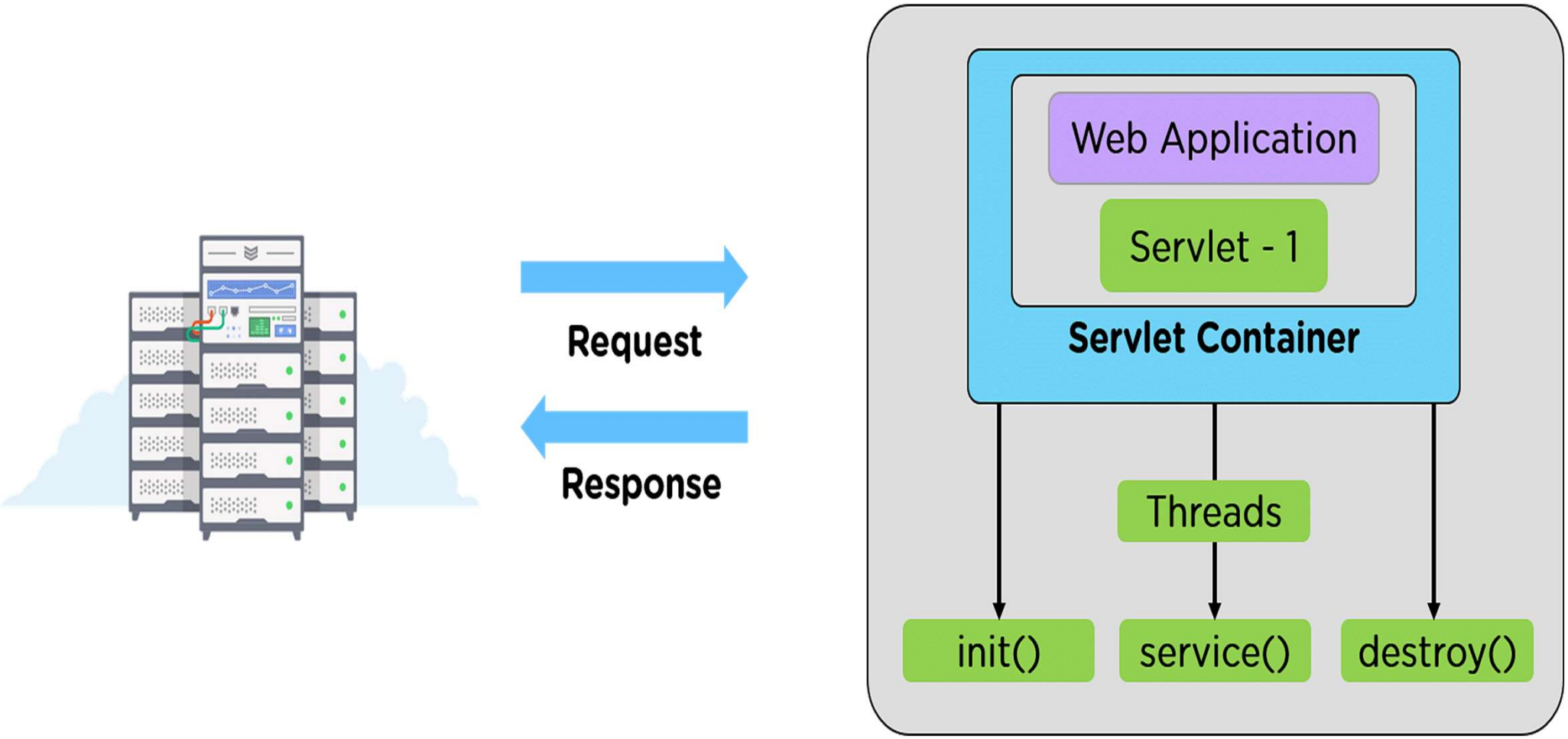
Architecture Diagram 2



Architecture Diagram 3



Architecture Diagram 4



Getting Parameters from the Request

- All HTTP requests can contain data.
- Data is in the form key=value
- All data and keys are of String type
- All HTTP data are encoded in a Hashtable and included with the request.
- If the servlet knows the names of the keys, it can request the value of the given key

```
String name = request.getParameter("name");
```

- If the key names are not know, the servlet can enumerate (iterate) through the keys and obtain data in that manner

```
Enumeration enum = request.getParameterNames();
while(enum.hasMoreElements()) {
    String key = (String) enum.nextElement();
    // …
```

A “Hello World” servlet

```
public class HelloServlet extends HttpServlet {
  public void doGet(HttpServletRequest request,
                    HttpServletResponse response)
      throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    String docType =
      "<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0 " + "Transitional//EN\">\n";
    out.println(docType +
                "<HTML>\n" +
                "<HEAD><TITLE>Hello</TITLE></HEAD>\n" +
                "<BODY BGCOLOR=\"#FDF5E6\">\n" +
                "<H1>Hello World</H1>\n" +
                "</BODY></HTML>");
  }
}
```

Benefits of servlet

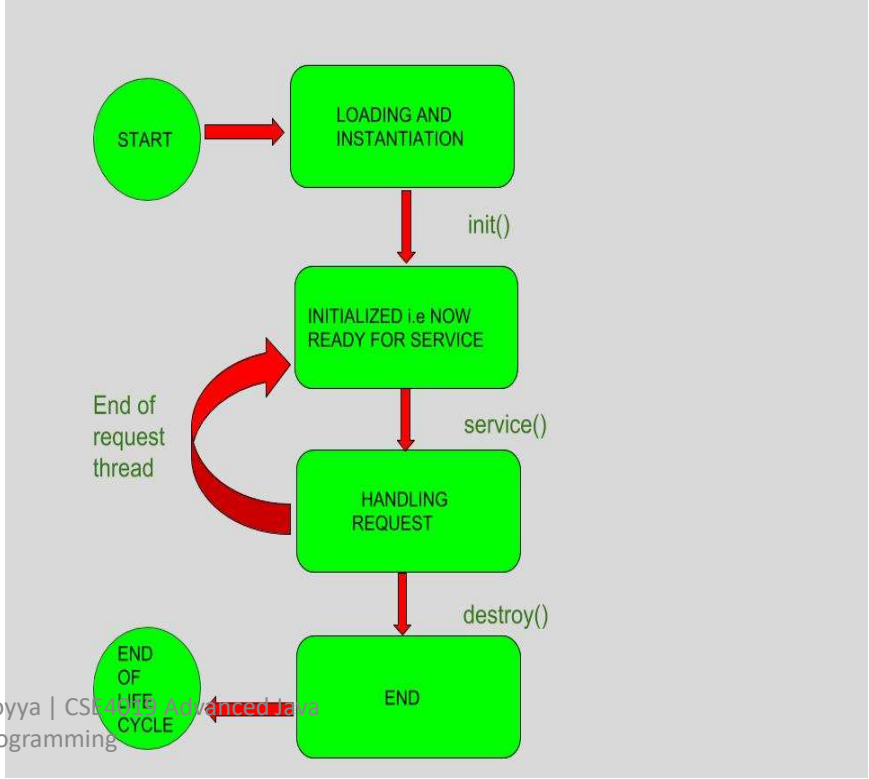
- Portability
- Powerful
- Efficiency
- Safety
- Integration
- Extensibility
- Inexpensive
- Secure
- Performance

Anatomy of a Servlet

- init()
- destroy()
- service()
  - doGet()
  - doPost()

Life Cycle of a Servlet

- **Stages of the Servlet Life Cycle**: The Servlet life cycle mainly goes through four stages,
- Loading a Servlet.
- Initializing the Servlet.
- Request handling.
- Destroying the Servlet.



Life Cycle of a Servlet

- In the life cycle of a servlet, we have mainly three stages, which are mentioned below.
- init()
- service()
- destroy()

Life Cycle of a Servlet

- Servlet API life cycle methods
  - init(): called when servlet is instantiated; must return before any other methods will be called
    - public void init() throws ServletException { //init() method initializing }
  - service(): method called directly by server when an HTTP request is received; default service() method calls doGet() (or related methods covered later)
    - GET, PUT, UPDATE, DELETE
      - doGet() for GET
      - doPut() for PUT
      - doUpdate() for UPDATE
      - doDelete() for DELETE
  - destroy(): called when server shuts down

Anatomy of a Servlet

- HTTPServletRequest object
  - Information about an HTTP request
    - Headers
    - Query String
    - Session
    - Cookies
- HTTPServletResponse object
  - Used for formatting an HTTP response
    - Headers
    - Status codes
    - Cookies

Server-side Programming

- The combination of
  - HTML
  - JavaScript
  - DOM
    is sometimes referred to as Dynamic HTML (DHTML)
- Web pages that include scripting are often called dynamic pages (vs. static)

Server-side Programming

- Similarly, web server response can be static or dynamic
  - Static: HTML document is retrieved from the file system and returned to the client
  - Dynamic: HTML document is generated by a program in response to an HTTP request
- Java servlets are one technology for producing dynamic server responses
  - Servlet is a class instantiated by the server to produce a dynamic response

Servlets vs. Java Applications

- Servlets do not have a main()
- The main() is in the server
- Entry point to servlet code is via call to a method (doGet() in the example)
- Servlet interaction with end user is indirect via request/response object APIs
- Actual HTTP request/response processing is handled by the server
- Primary servlet output is typically HTML

Methods

- Cookie[] getCookies()
- Enumeration getAttributeNames()
- Enumeration getHeaderNames()
- Enumeration getParameterNames()
- HttpSession getSession()
- HttpSession getSession(boolean create)
- Locale getLocale()
- Object getAttribute(String name)
- ServletInputStream getInputStream()
- String getAuthType()
- String getCharacterEncoding()
- String getContentType()
- String getContextPath()
- String getHeader(String name)
- String getMethod()
- String getParameter(String name)

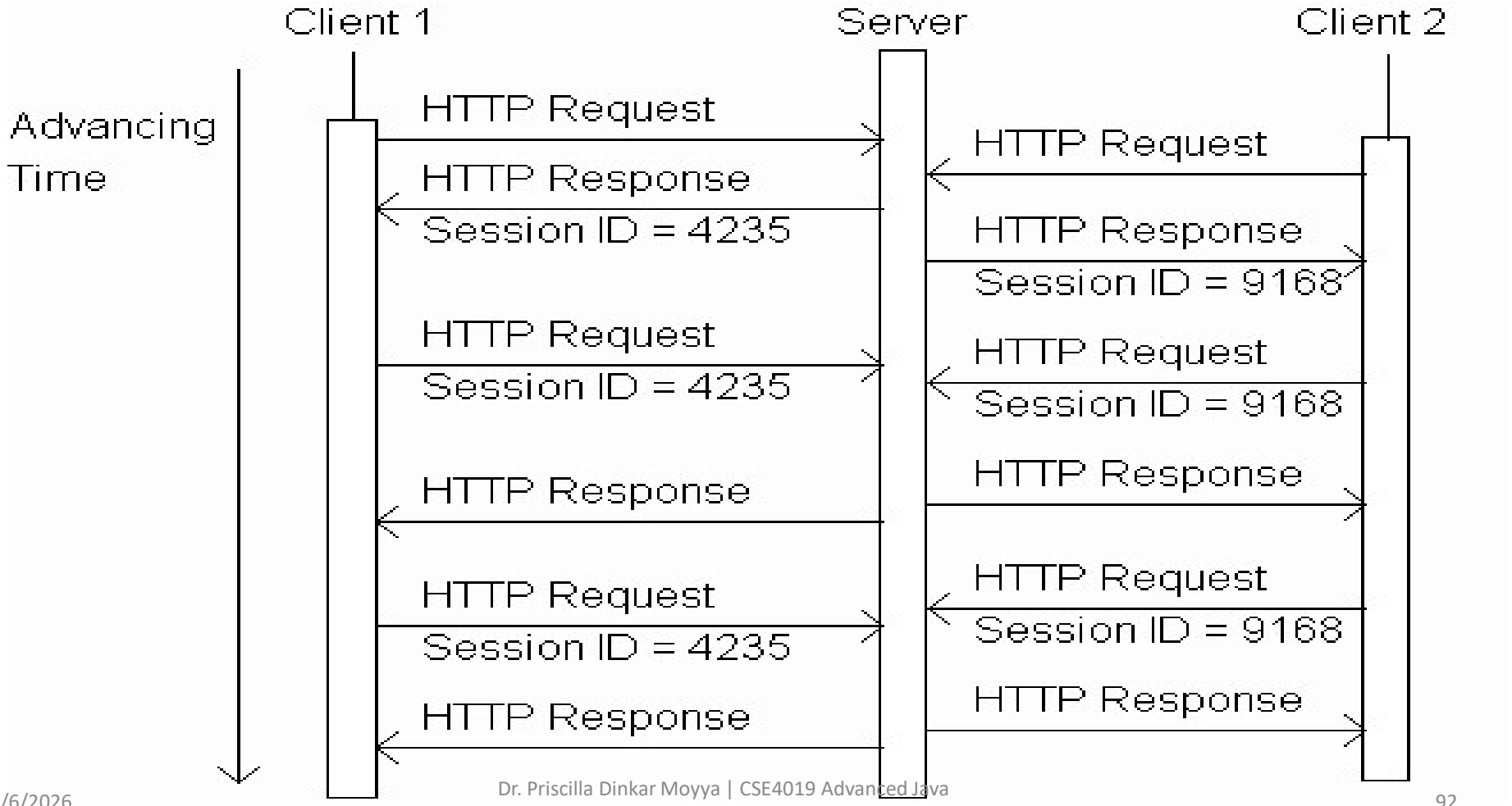
Sr.No.	Method & Description
1	**String encodeRedirectURL(String url)** Encodes the specified URL for use in the sendRedirect method or, if encoding is not needed, returns the URL unchanged.
2	**String encodeURL(String url)** Encodes the specified URL by including the session ID in it, or, if encoding is not needed, returns the URL unchanged.
3	**boolean containsHeader(String name)** Returns a Boolean indicating whether the named response header has already been set.
4	**boolean isCommitted()** Returns a Boolean indicating if the response has been committed.
5	**void addCookie(Cookie cookie)** Adds the specified cookie to the response.
6	**void addDateHeader(String name, long date)** Adds a response header with the given name and date-value.

Sr.No.	Method & Description
7	**void addHeader(String name, String value)** Adds a response header with the given name and value.
8	**void addIntHeader(String name, int value)** Adds a response header with the given name and integer value.
9	**void flushBuffer()** Forces any content in the buffer to be written to the client.
10	**void reset()** Clears any data that exists in the buffer as well as the status code and headers.
11	**void resetBuffer()** Clears the content of the underlying buffer in the response without clearing headers or status code.
12	**void sendError(int sc)** Sends an error response to the client using the specified status code and clearing the buffer.

Sessions

- Many interactive Web sites spread user data entry out over several pages:
  - Ex: add items to cart, enter shipping information, enter billing information
- Problem: how does the server know which users generated which HTTP requests?
  - Cannot rely on standard HTTP headers to identify a user

Sessions


Client 1
Server
Client 2
Advancing
Time
HTTP Request
HTTP Response
Session ID = 4235
HTTP Request
HTTP Response
Session ID = 9168
HTTP Request
Session ID = 4235
HTTP Request
Session ID = 9168
HTTP Response
HTTP Response
HTTP Request
Session ID = 4235
HTTP Request
Session ID = 9168
HTTP Response
HTTP Response

Sessions

```
HttpSession session = request.getSession();
if (session.isNew()) {
    visits++;
}
```

Returns HttpSession object associated with this HTTP request.

- Creates new `HttpSession` object if no session ID in request or no object with this ID exists
- Otherwise, returns previously created object

Sessions

```
public void doGet (HttpServletRequest request,
                   HttpServletResponse response)
        ,,,

HttpSession session = request.getSession();
String signIn = (String)session.getAttribute("signIn");
if (session.isNew() || (signIn == null)) {
    printSignInForm(servletOut, "Greeting");
} else {
    printWelcomeBack(servletOut, signIn);
}
```

Sessions

Second argument ("Greeting") used as `action` attribute value (relative URL)

```
private void printSignInForm(PrintWriter servletOut,
                             String action)
{
    ...
    servletOut.println(
"    <form method='post' action='" + action + "'><div> \n" +
"      <label> \n" +
"        Please sign in: <input type='text' name='signIn' /> \n" +

...
```

Sessions

```
public void doPost (HttpServletRequest request,
                    HttpServletResponse response)
            ...
String signIn = request.getParameter("signIn");
HttpSession session = request.getSession();
if (signIn != null) {
    printThanks(servletOut, signIn, "Greeting");
    session.setAttribute("signIn", signIn);
} else {
    printSignInForm(servletOut, "Greeting");
}
```

Cookies

- Servlets can set cookies explicitly
  - Cookie class used to represent cookies
  - request.getCookies() returns an array of Cookie instances representing cookie data in HTTP request
  - response.addCookie(Cookie) adds a cookie to the HTTP response

Cookies

TABLE 6.3: Key `Cookie` class methods.

Method	Purpose
`Cookie(String name, String value)`	Constructor to create a cookie with given name and value
`String getName()`	Return name of this cookie
`String getValue()`	Return value of this cookie
`void setMaxAge(int seconds)`	Set delay until cookie expires. Positive value is delay in seconds, negative value means that the cookie expires when the browser closes, and 0 means delete the cookie.

Cookies

```
public void doGet (HttpServletRequest request,
                   HttpServletResponse response)
    throws ServletException, IOException
{
    // Get count from cookie if available, otherwise
    // use initial value.
    int count = 0;
    Cookie[] cookies = request.getCookies();
    if (cookies != null) {
        for (int i=0; (i<cookies.length) && (count==0); i++) {
            if (cookies[i].getName().equals("COUNT")) {
                count = Integer.parseInt(cookies[i].getValue());
            }
        }
    }
```

EJB Enterprise Java Beans

https://www.youtube.com/watch?v=fOQz_m3jjPQ

https://examples.javacodegeeks.com/java-development/enterprise-java/ejb3/ejb-tutorial-beginners-example/

What is EJB

- EJB is an acronym for *enterprise java bean*. It is a specification provided by Sun Microsystems to develop secured, robust and scalable distributed applications.
- To run EJB application, you need an *application server* (EJB Container) such as Jboss, Glassfish, Weblogic, Websphere etc. It performs:
  a. life cycle management,
  b. security,
  c. transaction management, and
  d. object pooling.
- EJB application is deployed on the server, so it is called server side component also.
- EJB is like COM (*Component Object Model*) provided by Microsoft. But, it is different from Java Bean, RMI and Web Services.

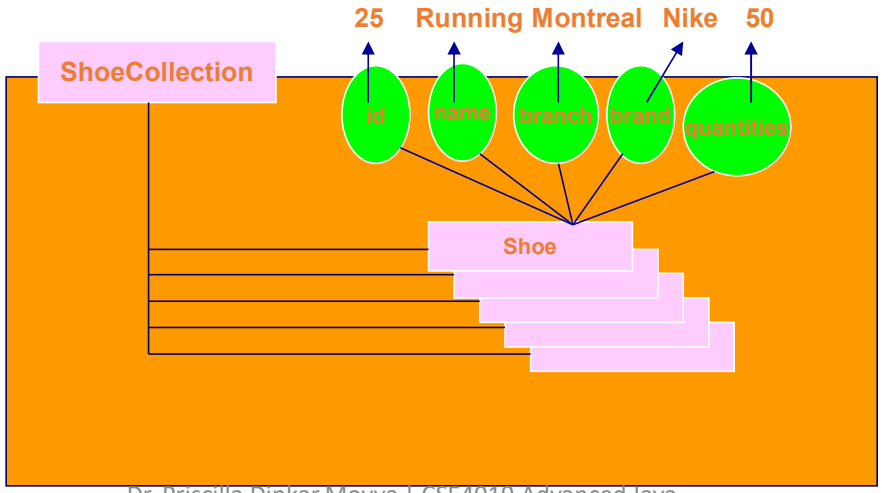
When use Enterprise Java Bean?

1. Application needs Remote Access. In other words, it is distributed.
2. Application needs to be scalable. EJB applications supports load balancing, clustering and fail-over.
3. Application needs encapsulated business logic. EJB application is separated from presentation and persistent layer.

Introduction

- EJB specification defines an architecture for the development and deployment of transactional, distributed object applications-based, server-side software components.

- Case Study
  - Shoe Retailer Company

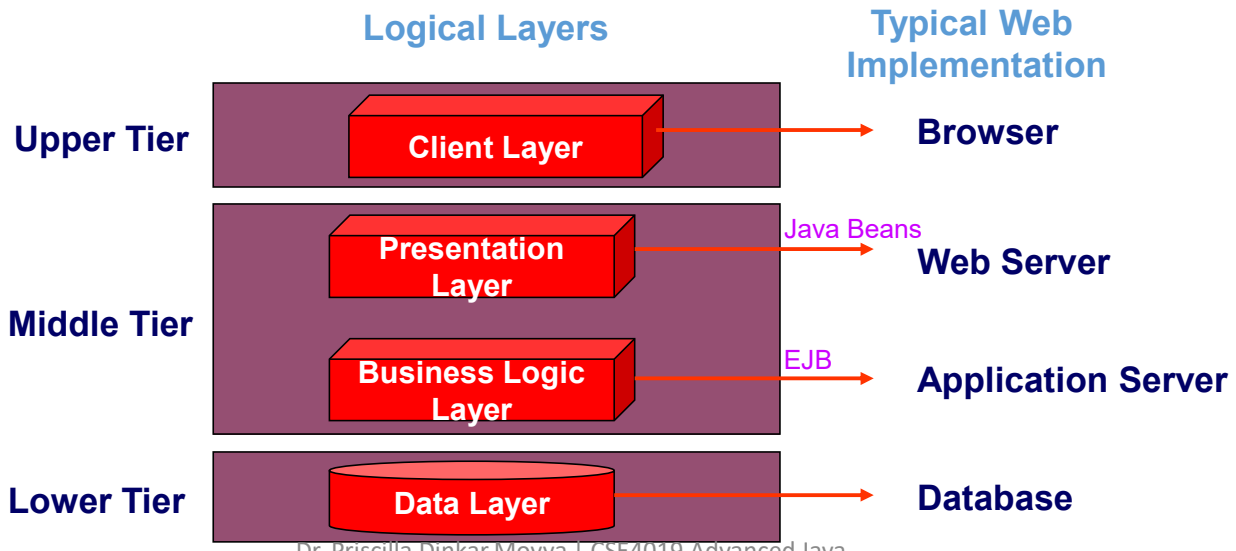


Comparison between JavaBeans and EJB

JavaBeans	Enterprise JavaBeans
JavaBeans may be visible or nonvisible at runtime.	An EJB is a non-visual, remote object.
JavaBeans are intended to be local to a single process and are primarily intended to run on the client side.	EJBs are remotely executable components or business objects that can be deployed only on the server.
JavaBeans is a component technology to create generic Java components that can be composed together into applets and applications.	Even though EJB is a component technology, it neither builds upon nor extends the original JavaBean specification.
JavaBeans are not typed.	EJBs are of two types—session beans and entity beans.
No explicit support exists for transactions in JavaBeans.	EJBs may be transactional and the EJB Servers provide transactional support.

Three-Tiered Architecture

- Client Layer
- Presentation Layer
- Business Logic Layer
- Data Layer



JBoss Application Server

- JBoss is an application server written in Java that can host EJB component.
- JBoss provides a container.
- An EJB container implicitly manages resources for EJB:
  - Threads
  - Socket
  - Database connections
  - Remote accessibility
  - Mutliclient support
  - Persistence management

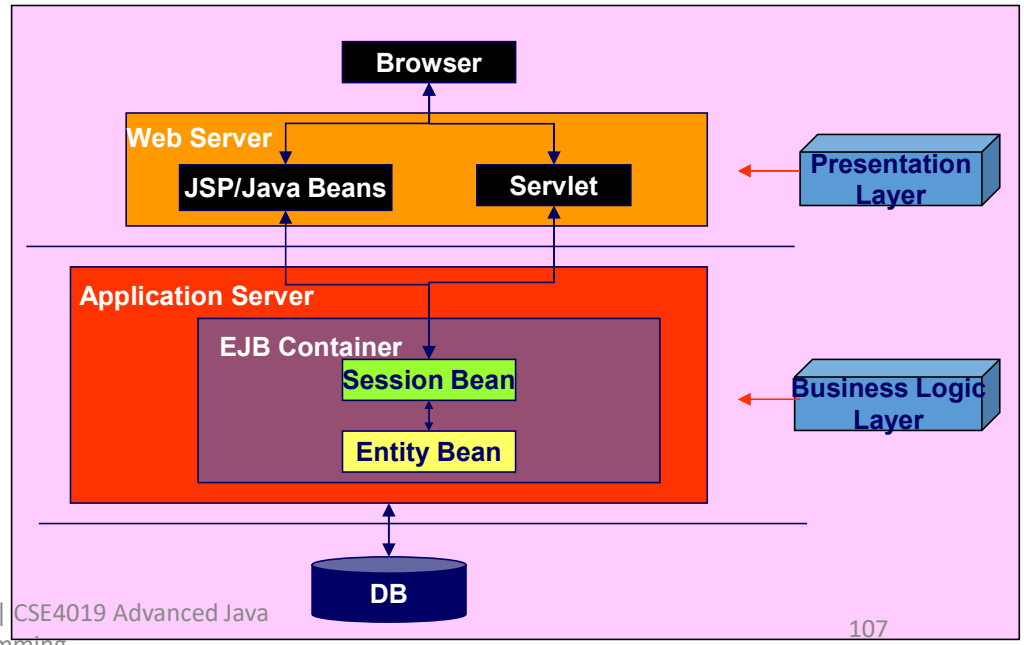
Enterprise Java Beans

Bean Types

Session Beans =>

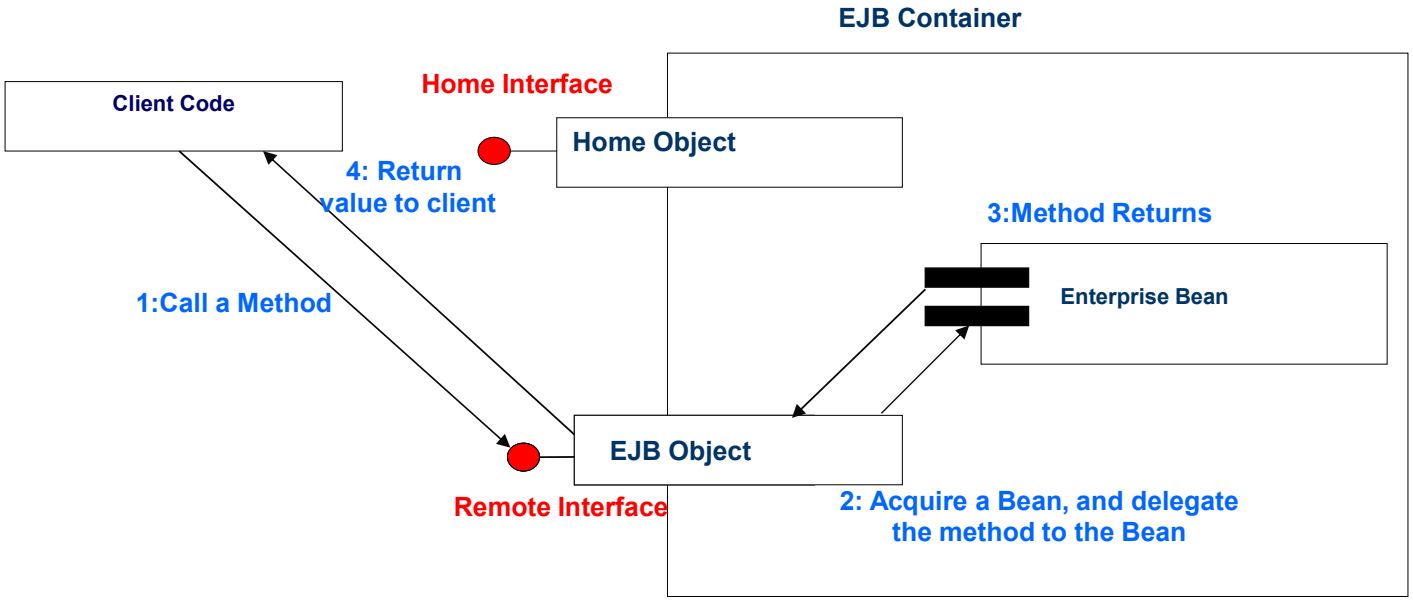
- models business processes
- Session bean contains business logic that can be invoked by local, remote or webservice client.

- **Entity Beans =>**
- models business data
- It encapsulates the state that can be persisted in the database. It is deprecated. Now, it is replaced with JPA (Java Persistent API).

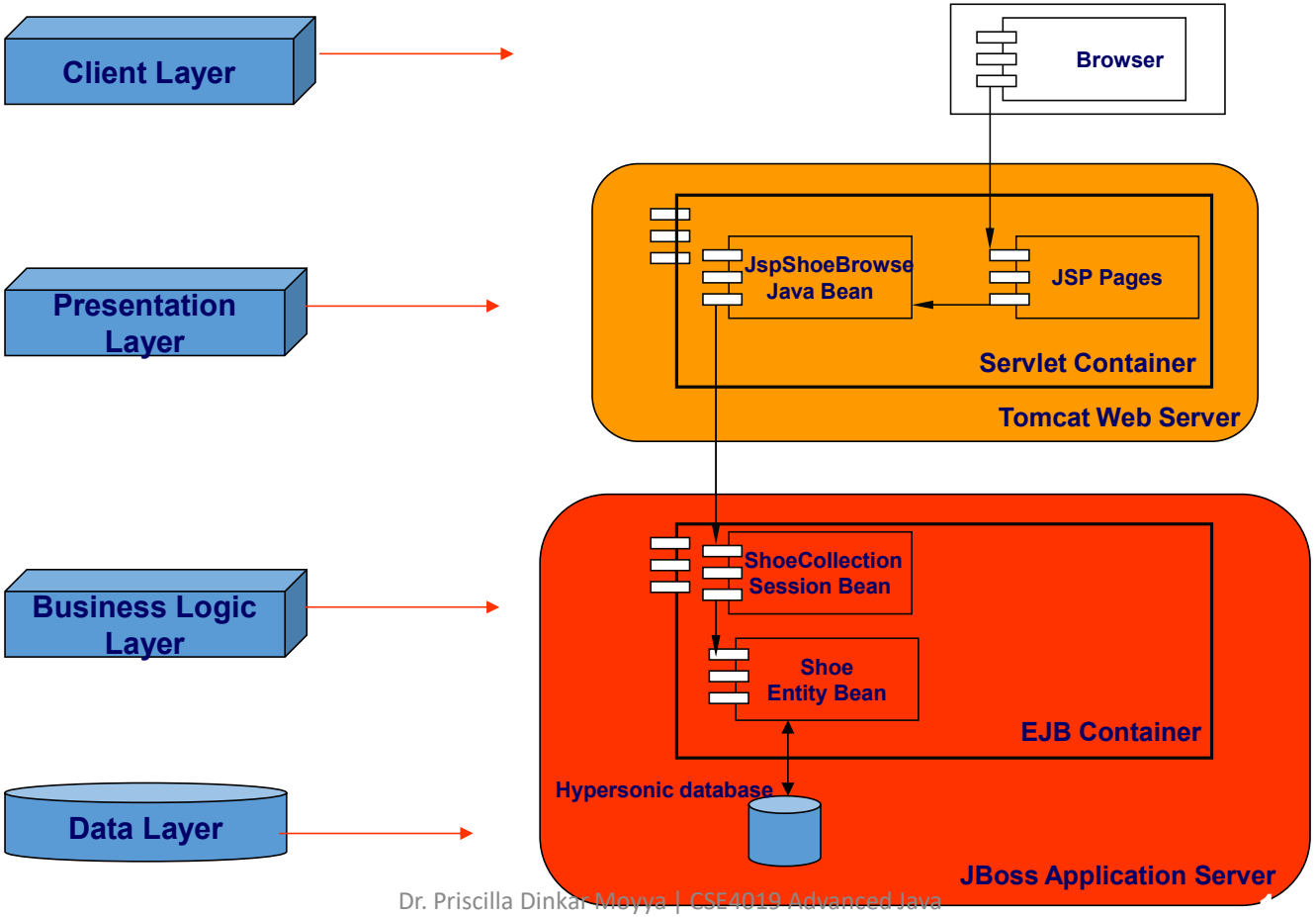


Accessing a Business Method

- Handling a client request to a business method.



Shoe Distributed Web Application



Difference between RMI and EJB

RMI	EJB
In RMI, middleware services such as security, transaction management, object pooling etc. need to be done by the java programmer.	In EJB, middleware services are provided by EJB Container automatically.
RMI is not a server-side component. It is not required to be deployed on the server.	EJB is a server-side component, it is required to be deployed on the server.
RMI is built on the top of socket programming.	EJB technology is built on the top of RMI.

Disadvantages of EJB

- Requires application server
- Requires only java client. For other language client, you need to go for webservice.
- Complex to understand and develop EJB applications.

Example of Stateless Session Bean

To develop stateless bean application, we are going to use **Eclipse IDE** and **glassfish 3** server.

To create EJB application, you need to create bean component and bean client.

1) Create stateless bean component

To create the stateless bean component, you need to create a remote interface and a bean class.

Example of Stateless Session Bean

File: AdderImplRemote.java

```
import javax.ejb.Remote;

@Remote
public interface AdderImplRemote {
int add(int a,int b);
}
```

File: AdderImpl.java

```
import javax.ejb.Stateless;

@Stateless(mappedName="st1")
public class AdderImpl implements AdderI
mplRemote {
  public int add(int a,int b){
     return a+b;
  }
}
```

Example of Stateless Session Bean

2) Create stateless bean client

The stateless bean client may be local, remote or webservice client. Here, we are going to create remote client. It is console based application. Here, we are not using dependency injection. The dependency injection can be used with web based client only.

Example of Stateless Session Bean

File: AdderImpl.java

Output

```
import javax.naming.Context;
import javax.naming.InitialContext;

public class Test {
public static void main(String[] args)throws Exception {
    Context context=new InitialContext();
    AdderImplRemote remote=(AdderImplRemote)context.lookup("st1");
    System.out.println(remote.add(32,32));
}
}
```

Output: 64

END